Layak
An Automatic Scholarship Application Agent

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

Email: umhackathon@um.edu.my

1. Introduction:

This System Analysis Documentation explains the technical design, architecture, data flow, and implementation scope of the Automatic Scholarship Application Agent. The product is based on the PRD, where the system is described as an AI-powered platform that helps students evaluate essays, check scholarship eligibility, match suitable scholarships, and manage their applications.

1.1. Purpose:

The purpose of this document is to describe how the system will be designed and developed from a technical perspective. It acts as a reference for developers, testers, and team members during implementation.

The key areas covered are:

1. ***Architecture**:*
   The system follows a client-server architecture using a React frontend, Python Flask backend, SQLAlchemy database layer, and a planned external AI service layer, using Gemini API.
2. ***Data Flows:***
   The document explains how student data, essays, scholarship requirements, and AI evaluation results move through the system.
3. ***Model Process:***
   The system model describes the flow for core features such as essay evaluation, eligibility checking, scholarship matching, and application tracking.
4. ***Role of Reference**:*
   This document provides a shared technical reference for the development team, frontend team, backend team, data handling team, and testing team.

1.2. Background:

Students applying for scholarships often need to repeat the same information across different applications, manually read long eligibility requirements, and rewrite essays for different scholarship criteria. This process is time-consuming and confusing.

The proposed system reduces this burden by centralising the application workflow. Students can store profile details, upload essays, check eligibility, receive feedback, and track application readiness in one platform.

1. *Previous Version:*

   There is no existing system version. The current project is designed as an MVP for UMHackathon 2026.

2. *Major Architectural Components*
   - *React frontend*
   - *Flask backend API*
   - *SQLAlchemy ORM database layer*
   - *AI service layer*
   - *Scholarship and application data storage*
   - *Dashboard and tracking system*

3. *New capabilities that are only introduced in this*
   - *Essay scoring and feedback*
   - *Eligibility checking*
   - *Scholarship matching*
   - *Content reshaping*
   - *Application readiness tracking*

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Students	*Use the platform to manage scholarship applications*	*Simple application workflow, useful feedback and eligibility checking*
Backend Developers	*Build API, database logic, and AI service layer*	*Clear API structure and modular backend design*
Frontend Developer	*Build a user interface and dashboard*	*Clear API responses and user-friendly data presentation*
Data Handler	*Manage scholarship data, profile data, and structured storage*	*Clean database schema and reliable data format*

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web application*
Frontend	*React*
Backend	*Python Flask*
Database Layer	*SQLAlchemy*
Architecture	*Client-server architecture*
AI Layer	*Planned Gemini API integration*

The system is designed as a web-based client-server application. The React frontend allows students to enter profile details, upload essays, view scholarship matches, and track application progress. The Flask backend handles authentication-related logic, profile management, essay processing, scholarship matching, and API communication.

SQLAlchemy is used as the database layer to manage structured data such as users, profiles, essays, scholarships, and applications. The AI service layer is planned as an external API integration, the Gemini API. This keeps the AI provider separate from the main backend logic, making the system easier to change if another model is used later.

2.1.2 LLM as Service Layer

The AI component is designed as a separate service layer instead of being directly embedded into frontend logic. The frontend does not call the AI API directly. Instead, the Flask backend prepares the request, validates the input, sends a structured prompt to the AI service, receives the response, parses it, and returns a clean result to the frontend.

Planned AI Service Flow:

1. *Student submits profile, essay, or selected scholarship.*

2. *React sends the request to the Flask backend.*
3. *Flask validates the input.*
4. *SQLAlchemy retrieves related profile, essay, and scholarship data.*
5. *Flask constructs a structured prompt.*
6. *The prompt is sent to the Gemini API.*
7. *Gemini returns essay feedback, eligibility result, or scholarship match reasoning.*
8. *Flask parses the response into structured output.*
9. *The result is stored in the database if needed.*
10. *React displays the result to the student.*

Context Sent to AI:

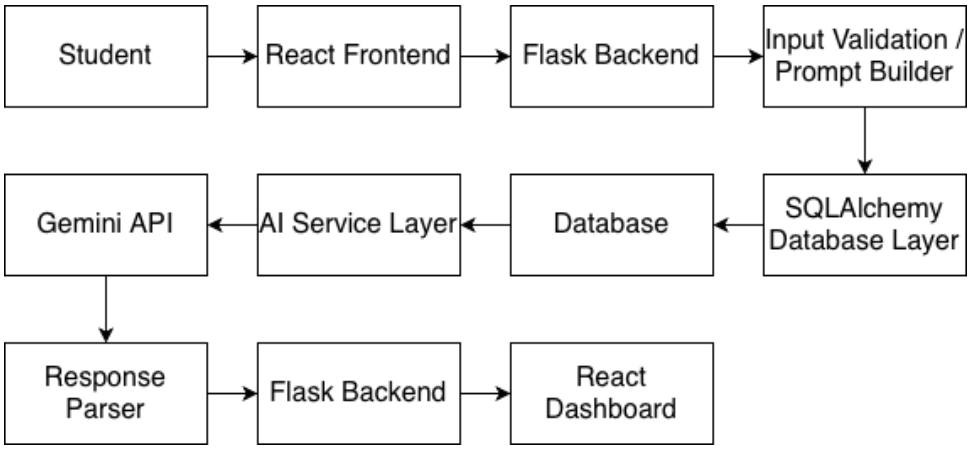
Depending on the feature, the AI context may include:

- *Student profile*
- *Academic details*
- *Essay content*
- *Scholarship requirements*
- *Eligibility rules*
- *Word limits*
- *Previous essay version*
- *Selected application goal*

Token and Input Limitation Handling

Before sending content to the AI service, the backend will enforce input limits. Long essays or large scholarship descriptions may be shortened, chunked, or summarised before being included in the prompt. This prevents unnecessary token usage and reduces API cost.

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram

a. *Student → React Frontend: Upload essay*

b. *React Frontend → Flask Backend: POST essay evaluation request*

c. *Flask Backend → Database: Retrieve student profile and essay details*

d. *Database → Flask Backend: Return stored data*

e. *Flask Backend → Prompt Builder: Prepare structured evaluation prompt*

f. *Prompt Builder → Gemini API: Send essay + criteria*

g. *Gemini API → Flask Backend: Return score and feedback*

h. *Flask Backend → Response Parser: Validate and structure response*

i. *Flask Backend → Database: Store evaluation result*

j. *Flask Backend → React Frontend: Return feedback*

k. *React Frontend → Student: Display score, weak sections, and suggestions*

The student uploads an essay from the frontend. The Flask backend receives the essay, retrieves the student profile and relevant scholarship criteria, then sends a structured prompt to the AI service. After the AI returns feedback, the backend validates and stores the response before sending it back to the dashboard.

2.2. Technological Stack

2.2.1. Frontend uses React as it is suitable for building interactive dashboards and form-based user interfaces

2.2.2. Backend uses Python Flask as it is lightweight and fast for MVP API development

2.2.3. Database uses SQLAlchemy as it simplifies database interaction using Python ORM

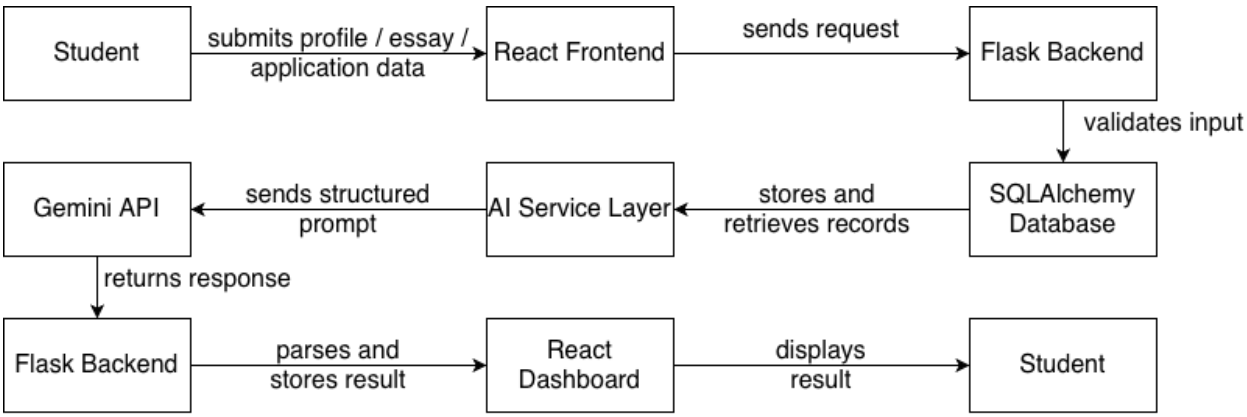
2.2.4. AI services uses Gemini API for essay evaluation, eligibility reasoning, and scholarship matching

2.3. Key Data Flows

2.3.1. How data are moving through the system

2.3.1.1. Data Flow Diagram (DFD)

Email: umhackathon@um.edu.my



Student information is first collected through the React frontend. The Flask backend validates the data and stores it using SQLAlchemy. When an AI-powered feature is requested, the backend retrieves the required profile, essay, or scholarship information and prepares a structured AI prompt. The AI response is parsed before being displayed to the user.

2.3.2. **Normalized Database Schema**

users	*Stores account login details such as email, password hash, and creation date.*
profiles	*Stores the student's personal, academic, financial, and background information.*
universities	*Stores university information linked to scholarships.*
scholarships	*Stores scholarship details, eligibility rules, essay prompts, required documents, and evaluation criteria.*
essays	*Stores student essays, including essay versions and timestamps.*
essay_evaluations	*Stores AI-generated essay scores, matched requirements, weaknesses, and revision suggestions.*
eligibility_results	*Stores whether a student is eligible for a scholarship and which criteria they passed or failed.*

scholarship_matches	*Stores scholarship matching results, including fit score, match reason, and risk notes.*
reshaped_contents	*Stores AI-reshaped essay content based on another scholarship question or word limit.*
applications	*Tracks each student's scholarship application status, readiness score, latest score, deadline, and linked essay.*
documents	*Stores uploaded supporting documents such as certificates, transcripts, or identification files.*
notifications	*Stores system notifications such as deadline reminders or application updates.*

users 1 —— 1 profiles

universities 1 —— M scholarships

users 1 —— M essays

essays 1 —— M essay_evaluations
scholarships 1 —— M essay_evaluations

users 1 —— M eligibility_results
scholarships 1 —— M eligibility_results

users 1 —— M scholarship_matches
scholarships 1 —— M scholarship_matches

users 1 —— M reshaped_contents
essays 1 —— M reshaped_contents
scholarships 1 —— M reshaped_contents

users 1 —— M applications
scholarships 1 —— M applications
essays 1 —— M applications

users 1 —— M documents

users 1 —— M notifications

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. These MVP features are based on the core features listed in the PRD, including essay feedback, eligibility checking, matching, content reshaping, and application tracking.

#	Features	Description
1	Student Profile Management	Students can enter and update personal, academic, and background details
2	Essay Scoring and Feedback	Students upload essays and receive structured feedback
3	Eligibility Checker	The system compares student profile data with scholarship requirements
4	Smart Scholarship Matching	The system recommends scholarships based on student profile and fit
5	Application Tracking Dashboard	Students can view application readiness, status, and deadlines

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality	Requirements	Implementation
Usability	Students should be able to submit profile and essay data easily	Use a simple React dashboard with clear forms
Reliability	The system should avoid showing broken or incomplete AI responses	Validate AI response before displaying it
Maintainability	Backend logic should be modular	Separate Flask routes, database models, and AI service functions

Scalability	*The system should support more users and scholarships in the future*	*Use structured database tables and reusable APIs*
Token Latency	*AI responses should return within an acceptable wait time*	*Use loading states and backend timeout handling*
Cost Efficiency	*API calls should be controlled*	*Limit input size, cache repeated requests, and avoid unnecessary AI calls*
Data Integrity	*Student, essay, and scholarship data should remain consistent*	*Use relational schema and SQLAlchemy models*
Security	*API keys should not be exposed to the frontend*	*Store AI API key only in backend environment variables*

3.3. Out of Scope / Future Enhancements

The following features are not included in the UMHackathon:

- *AI writing full essays from scratch*
- *Full interview coaching simulation*
- *Multi-user collaboration*
- *Production-level testing features*
- *Advanced frontend UI systems*
- *Payment integration*
- *Real-time chat with advisors*
- *Direct submission to scholarship providers*

These are intentionally excluded because the MVP focuses on evaluation, guidance, matching, and tracking rather than replacing the student's own application work.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

For the MVP, the system can be tested using a small group of users first. The first rollout should focus on essay evaluation and eligibility checking because these are the most important AI-assisted features.

A possible rollout plan:

Stage	***Description***
Internal Testing	*Team members test with sample essays and scholarship criteria*
Small User Testing	*A small group of students test profile, essay, and eligibility features*
Feedback Collection	*Collect issues related to wrong scoring, confusing output, or missing requirements*
Improvement Stage	*Improve prompts, validation logic, and user interface*
Full Demo	*Present stable MVP during hackathon demo*

A/B testing can be used for essay feedback format. For example, one version may show feedback as bullet points, while another version may show feedback grouped by criteria such as clarity, relevance, structure, and grammar.

4.1.2. ***Emergency Rollback (ER) & Golden Release****:*

A stable version of the system should be maintained as the Golden Release. If a new update causes major failures, the team can roll back to the previous working version.

Rollback should be triggered when:

- *Frontend cannot connect to backend*
- *Database operations fail*
- *AI response parsing fails repeatedly*
- *Core demo features stop working*
- *Testing breaks before submission*

4.1.3. ***Priority Matrix:***

4.1.3.1.

Priority	***Trigger Condition***	***Action***
P1 Critical	*Login, dashboard, essay evaluation, or eligibility check completely fails*	*Roll back to Golden Release immediately*

P2 High	*AI gives unusable or malformed responses repeatedly*	*Disable AI result display and show fallback message*
P3 Medium	*Some results are delayed or incomplete*	*Show loading state and allow retry*
P4 Low	*Minor UI formatting issue*	*Fix in next update cycle*

4.2. Monitoring:

4.2.1. The system should monitor both technical and AI-related behaviour.

Monitoring Areas

Area	***What to monitor***	***Reason***
Backend API	*Request failure rate*	*Detect broken routes or server issues*
Database	*Failed inserts or missing records*	*Prevent data loss*
AI API	*Timeout, malformed response, rate limit*	*Ensure AI results are usable*
Frontend	*Form errors and failed requests*	*Improve user experience*
Cost	*Number of AI calls*	*Control API usage*

Example Health Checks:

- */health endpoint for backend status*
- *Database connection check*
- *AI API availability check*
- *Logging failed AI calls*
- *Tracking average response time*

4.3. Assumptions:

The system is developed based on the following assumptions:

- *Students have access to a stable internet connection.*
- *Students provide truthful and complete profile information.*
- *Scholarship requirement data is available in structured or semi-structured form.*

- *The Gemini API key is available during implementation.*
- *AI-generated feedback must be reviewed by the user before being trusted fully.*
- *The system is used as a support tool, not a final decision-maker.*
- *The MVP does not directly submit applications to scholarship providers.*

4.4. External Dependencies: *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	*Purpose*	*Risks*	*Mitigation*
Gemini API	*Planned AI model for essay feedback, eligibility reasoning, and matching*	*API limit, latency, or unavailable service*	*Add fallback message and retry logic*
React	*Frontend user interface*	*UI bugs may affect user experience*	*Test main user flows*
Flask	*Backend API*	*Backend failure affects all core features*	*Maintain modular routes and health checks*
SQLAlchemy	*Database ORM*	*Incorrect schema design may cause data issues*	*Use clear model relationships*
Hosting Platform	*testing*	*Testing failure before demo*	*Keep local working version and Golden Release*

5. Project Management & Team Contributions

5.1. Project Timeline:

Timeline	***task***
Day 1–2	*Finalise product idea, PRD, and system scope*
Day 3-4	*Design architecture, database schema, and core backend routes*
Day 5-6	*Build frontend pages and connect to backend*

	APIs
Day 7	*Implement database models and data handling*
Day 8	*Integrate planned AI service layer / prepare mock AI response fallback*
Day 9	*Testing, debugging, and documentation*

5.2. Team Members Role:

Team member	***Role***	***Responsibility***
Ibaad	*Backend Developer*	*Flask API development and backend logic*
Jess	*Backend Developer*	*Backend routes, AI service layer, and system logic*
Subhan	*Data Handling*	*Database structure, scholarship data, and data processing*
Nadya	*Frontend Developer*	*React frontend, dashboard, and user interface*
Jessica	*Testing*	*Testing setup, environment configuration, and final system release*

5.3. Recommendations:

To improve scalability, reliability, and maintainability, the following recommendations are suggested:

- *Keep the AI service layer separate from backend routes so the model provider can be changed later.*
- *Store AI API keys in environment variables only.*
- *Use caching for repeated eligibility checks or scholarship matching results.*
- *Add input validation before sending data to the AI API.*
- *Maintain a Golden Release version before testing new features.*
- *Use structured JSON-like AI outputs to make parsing easier.*
- *Add clear fallback messages when AI output is unavailable or unreliable.*